

Assignment – 15.5

Name: Alli Harika

Batch: 21

HT.No: 2303A510I7

Task 1 – Student Attendance API

Task:

Create a RESTful API for student attendance tracking.

Instructions:

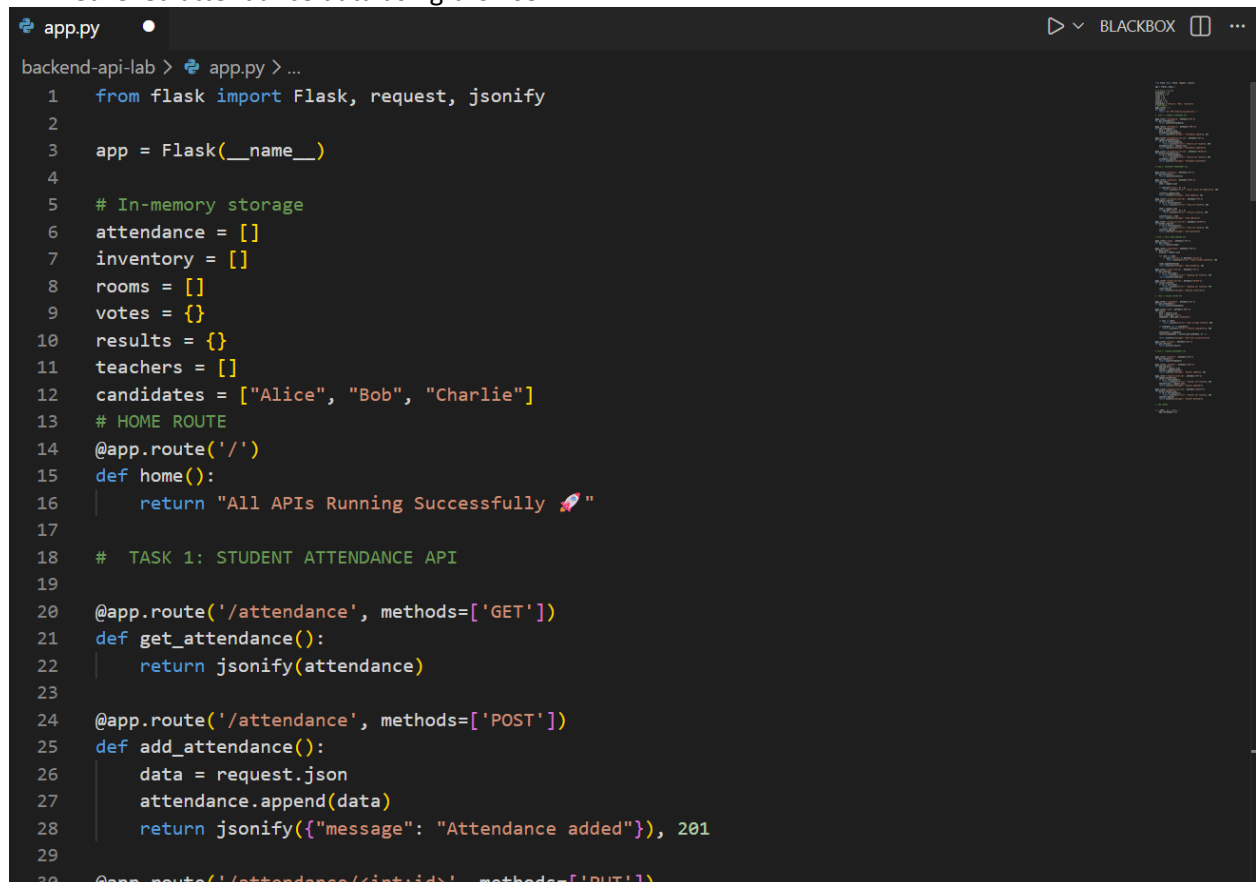
- Endpoints required:
 - o GET /attendance → View attendance records
 - o POST /attendance → Mark attendance
 - o PUT /attendance/{id} → Update attendance record
 - o DELETE /attendance/{id} → Remove attendance entry
- Ensure JSON-based request and response handling.

Expected Output:

- API for maintaining and updating attendance records.

Prompt:

- Created REST API for student attendance using Flask
- Implemented endpoints for POST and GET requests
- Used PowerShell to send JSON data
- Retrieved attendance data using browser



```
app.py
backend-api-lab > app.py > ...
1  from flask import Flask, request, jsonify
2
3  app = Flask(__name__)
4
5  # In-memory storage
6  attendance = []
7  inventory = []
8  rooms = []
9  votes = {}
10 results = {}
11 teachers = []
12 candidates = ["Alice", "Bob", "Charlie"]
13 # HOME ROUTE
14 @app.route('/')
15 def home():
16     return "All APIs Running Successfully 🚀"
17
18 # TASK 1: STUDENT ATTENDANCE API
19
20 @app.route('/attendance', methods=['GET'])
21 def get_attendance():
22     return jsonify(attendance)
23
24 @app.route('/attendance', methods=['POST'])
25 def add_attendance():
26     data = request.json
27     attendance.append(data)
28     return jsonify({"message": "Attendance added"}), 201
29
30 @app.route('/attendance/<int:id>', methods=['PUT'])
```

```

17
18 # TASK 1: STUDENT ATTENDANCE API
19
20 @app.route('/attendance', methods=['GET'])
21 def get_attendance():
22     return jsonify(attendance)
23
24 @app.route('/attendance', methods=['POST'])
25 def add_attendance():
26     data = request.json
27     attendance.append(data)
28     return jsonify({"message": "Attendance added"}), 201
29
30 @app.route('/attendance/<int:id>', methods=['PUT'])
31 def update_attendance(id):
32     if id >= len(attendance):
33         return jsonify({"error": "Record not found"}), 404
34     attendance[id] = request.json
35     return jsonify({"message": "Attendance updated"})
36
37 @app.route('/attendance/<int:id>', methods=['DELETE'])
38 def delete_attendance(id):
39     if id >= len(attendance):
40         return jsonify({"error": "Record not found"}), 404
41     attendance.pop(id)
42     return jsonify({"message": "Attendance deleted"})
43

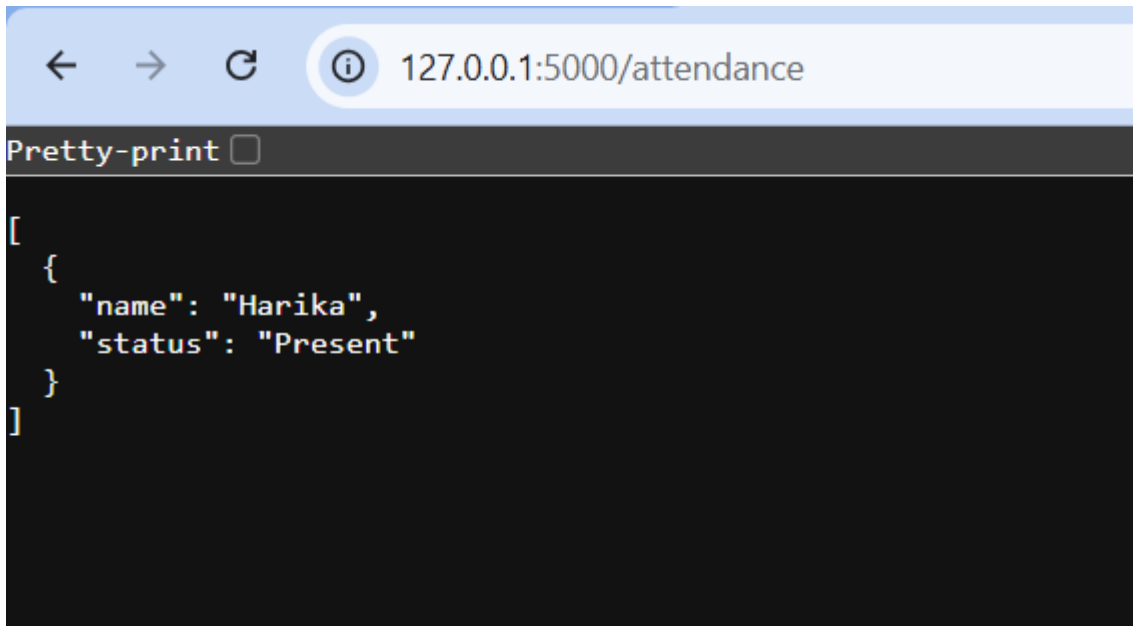
```

pretty-print ☐

```

{
  "name": "Harika",
  "status": "Present"
},
{
  "name": "Rahul",
  "status": "Absent"
}

```

A screenshot of a web browser window. The address bar shows the URL '127.0.0.1:5000/attendance'. Below the address bar, there is a 'Pretty-print' checkbox. The main content area displays a JSON array containing one object:

```
[
  {
    "name": "Harika",
    "status": "Present"
  }
]
```

Observation:

- Attendance record was successfully added
- Data stored in JSON format
- GET request displayed list of students
- API responded correctly without errors

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

- Endpoints required:
 - o GET /inventory → List all items
 - o POST /inventory → Add new item
 - o PUT /inventory/{id} → Update stock quantity
 - o DELETE /inventory/{id} → Remove an item
- Implement validation for stock values.

Expected Output:

- API capable of managing inventory stock levels.

Prompt:

- Developed API for inventory management
- Implemented POST, GET, PUT, DELETE operations
- Added item with stock value
- Updated and deleted inventory items

```

backend-api-lab > app.py > ...
44
45 # TASK 2: INVENTORY MANAGEMENT API
46 @app.route('/inventory', methods=['GET'])
47 def get_inventory():
48     return jsonify(inventory)
49
50 @app.route('/inventory', methods=['POST'])
51 def add_item():
52     item = request.json
53
54     if item.get("stock", 0) < 0:
55         return jsonify({"error": "Stock cannot be negative"}), 400
56
57     inventory.append(item)
58     return jsonify({"message": "Item added"}), 201
59
60 @app.route('/inventory/<int:id>', methods=['PUT'])
61 def update_item(id):
62     if id >= len(inventory):
63         return jsonify({"error": "Item not found"}), 404
64
65     item = request.json
66     if item.get("stock", 0) < 0:
67         return jsonify({"error": "Invalid stock"}), 400
68
69     inventory[id] = item
70     return jsonify({"message": "Item updated"})
71
72 @app.route('/inventory/<int:id>', methods=['DELETE'])
73 def delete_item(id):

```

Output:

```

(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http://127.0.0.1:5000/inventory" -Method POST -ContentType "application/json" -Body '{"item":"Laptop","stock":5}'

```

message

Item added

```

(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab>

```

```
Pretty-print ☐
[
  {
    "item": "Laptop",
    "stock": 5
  }
]
```

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http://127.0.0.1:5000/inventory/0" -Method PUT -ContentType "application/json" -Body '{"stock":10}'

message
-----
Item updated
```

```
Pretty-print ☐
[
  {
    "stock": 10
  }
]
```

```
Pretty-print ☐
[ ]
```

Observation:

- Item added successfully with given stock
- Stock value updated correctly
- Deleted item removed from list
- API handled CRUD operations properly

Task 3 – Hotel Room Booking API

Task:

Develop a RESTful API for hotel room booking.

Instructions:

- Endpoints required:
 - o GET /rooms → View available rooms
 - o POST /rooms/book → Book a room
 - o GET /rooms/{id} → View booking details
 - o DELETE /rooms/{id} → Cancel booking
- Implement error handling for unavailable rooms.

Expected Output:

- API simulating hotel room booking operations.

Prompt:

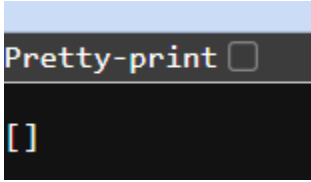
- Created API for hotel room booking
- Implemented endpoints for booking, viewing, and cancelling rooms
- Used JSON input for booking details

```
# TASK 3: HOTEL ROOM BOOKING API
@app.route('/rooms', methods=['GET'])
def get_rooms():
    return jsonify(rooms)
@app.route('/rooms/book', methods=['POST'])
def book_room():
    booking = request.json
    for room in rooms:
        if room['room_no'] == booking['room_no']:
            return jsonify({"error": "Room already booked"}), 400
    rooms.append(booking)
    return jsonify({"message": "Room booked"}), 201
@app.route('/rooms/<int:id>', methods=['GET'])
def get_room(id):
    if id >= len(rooms):
        return jsonify({"error": "Booking not found"}), 404
    return jsonify(rooms[id])
@app.route('/rooms/<int:id>', methods=['DELETE'])
def cancel_room(id):
    if id >= len(rooms):
        return jsonify({"error": "Booking not found"}), 404
    rooms.pop(id)
    return jsonify({"message": "Booking cancelled"})
```

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http://127.0.0.1:5000/rooms/0" -Method DELETE

message
-----
Booking cancelled

(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab>
```



Observation:

- Room booking was successful
- Booking details retrieved correctly
- Booking cancellation worked properly
- API handled requests without errors

Task 4 – Online Voting API

Task:

Create a RESTful API for an online voting system.

Instructions:

- Endpoints required:
 - o GET /candidates → List candidates
 - o POST /vote → Cast a vote
 - o GET /results → View voting results
- Ensure one vote per user using in-memory validation.

Expected Output:

- API simulating a secure online voting process.

Prompt:

- Developed API for online voting system
- Implemented endpoints for candidates, voting, and results
- Ensured one vote per user

```
# TASK 4: ONLINE VOTING API
@app.route('/candidates', methods=['GET'])
def get_candidates():
    return jsonify(candidates)
@app.route('/vote', methods=['POST'])
def vote():
    data = request.json
    user = data.get("user")
    candidate = data.get("candidate")
    if user in votes:
        return jsonify({"error": "User already voted"}), 400
    if candidate not in candidates:
        return jsonify({"error": "Invalid candidate"}), 400
    votes[user] = candidate
    results[candidate] = results.get(candidate, 0) + 1
    return jsonify({"message": "Vote cast successfully"})

@app.route('/results', methods=['GET'])
def get_results():
    return jsonify(results)
```

Pretty-print ☐

```
[
  "Alice",
  "Bob",
  "Charlie"
]
```

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http://127.0.0.1:5000/vote" -Method POST -ContentType "application/json" -Body '{"user":"harika123","candidate":"Alice"}'
```

```
message
-----
Vote cast successfully
```



```
Pretty-print ☐
{
  "Alice": 1
}
```

```
Pretty-print ☐
[]
```

Observation:

- Candidates list displayed correctly
- Vote cast successfully
- Results updated dynamically
- Duplicate voting was prevented

Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage teacher records.

Instructions:

- Endpoints required:
 - o GET /teachers → List all teachers
 - o POST /teachers → Add a new teacher
 - o PUT /teachers/{id} → Update teacher details
 - o DELETE /teachers/{id} → Remove a teacher
- Ensure proper error handling and JSON responses.

Expected Output:

- Functional API for managing teacher information. ...

Prompt:

- Created API for managing teacher records
- Implemented CRUD operations
- Added, updated, and deleted teacher details

```

# TASK 5: TEACHER MANAGEMENT API

@app.route('/teachers', methods=['GET'])
def get_teachers():
    return jsonify(teachers)

@app.route('/teachers', methods=['POST'])
def add_teacher():
    teacher = request.json
    teachers.append(teacher)
    return jsonify({"message": "Teacher added"}), 201

@app.route('/teachers/<int:id>', methods=['PUT'])
def update_teacher(id):
    if id >= len(teachers):
        return jsonify({"error": "Teacher not found"}), 404
    teachers[id] = request.json
    return jsonify({"message": "Teacher updated"})

@app.route('/teachers/<int:id>', methods=['DELETE'])
def delete_teacher(id):
    if id >= len(teachers):
        return jsonify({"error": "Teacher not found"}), 404
    teachers.pop(id)
    return jsonify({"message": "Teacher deleted"})

# RUN SERVER
if __name__ == '__main__':
    app.run(debug=True)

```

Output:

```

(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http://127.0.0.1:5000/teachers" -Method POST -ContentType "application/json" -Body '{"name":"Dr. Rao","subject":"AI"}'

```

```

message
-----
Teacher added

```

```

(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab>

```

pretty-print ☐

```

{
  "name": "Dr. Rao",
  "subject": "AI"
}

```

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http://127.0.0.1:5000/teachers/0" -Method PUT -ContentType "application/json" -Body '{"subject":"ML"}'  
  
message  
-----  
Teacher updated
```

Pretty-print ☐

```
[  
  {  
    "subject": "ML"  
  }  
]
```

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http://127.0.0.1:5000/teachers/0" -Method DELETE  
  
message  
-----  
Teacher deleted
```

Pretty-print ☐

```
[]
```

Observation:

- Teacher added successfully
- Data retrieved correctly
- Update operation modified teacher details
- Delete operation removed teacher record